

5	3	5	6	7	8	9		2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

(1) The digit 1-9 → once in any row
 (2) " " " " → " " any col
 (3) " " " " → " " any 3x3

False

5	3	1	6	7	8	9		2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3	4	6	7	8	9		2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

$$fnc(i = \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} \rightarrow 9)$$

7 (possible)

g(1)

L15. Sudoku Solver | Backtracking

1.50

↓

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

↙

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

↙

True

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	2	4	8	5	6
9	6	1	5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

L15. Sudoku Solver | Backtracking

1.50

8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9	4	8	5	6	
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

$$for(i = 1 \rightarrow 9)$$

$$if(possible)$$

$$f(i)$$

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3

5	3	5	6	7	8	9		2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

(1) The digit 1-9 → once in any row
 (2) " " " " → " " any col
 (3) " " " " → " " any 3x3

False

5	3	1	6	7	8	9		2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3	4	6	7	8	9		2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

5	3	4	6	7	8	9	1	2
6	7	2	1	9	5	3	4	8
	9	8	3	4	2	5	6	7
8	5	9	7	6	1	4	2	3
4	2	6	8	5	3	7	9	1
7	1	3	9		4	8	5	6
9	6		5	3	7	2	8	4
2	8	7	4	1	9	6	3	5
3	4	5	2	8	6	1	7	9

$$fnc(i = \begin{matrix} 1 \\ 2 \\ 3 \end{matrix} \rightarrow 9)$$

7 (possible)

g(1)

	0	1	2	3	4	5	6	7	8
0	5	3	.	6	7	8	9	1	2
1	6	7	2	1	9	5	3	4	8
2	.	9	8	3	4	2	5	6	7
3	8	5	9	7	6	1	4	2	3
4	4	2	6	8	5	3	7	9	1
5	7	1	3	9	.	4	8	6	5
6	9	6	.	5	3	7	2	8	4
7	2	8	7	4	1	9	6	3	5
8	3	4	5	2	8	6	1	7	9

$\text{for } i = 0 \rightarrow 8$
 if $\text{board}[\text{row}][i] == 1$ return false
 if $\text{board}[i][\text{col}] == 1$ return false

$$\text{row} = \left\lceil 3 \times \left(\frac{5-1}{3} \right) \right\rceil = 3 + \frac{4}{3} = 4$$

$$\text{col} = \left\lceil 3 \times \left(\frac{7-1}{3} \right) \right\rceil = 6 + \frac{2}{3} = 6$$

```

class Solution {
public:
    void solveSudoku(vector<vector<char>>& board) {
        solve(board);
    }

    bool solve(vector<vector<char>>& board){
        for(int i = 0; i < board.size(); i++) {
            for(int j = 0; j < board[0].size(); j++) {

                if(board[i][j] == '.'){

                    for(char c = '1'; c <= '9'; c++) {
                        if(isValid(board, i, j, c)){
                            board[i][j] = c;

                            if(solve(board) == true)
                                return true;
                            else
                                board[i][j] = '.';
                        }
                    }

                    return false;
                }
            }
        }

        return true;
    }

    bool isValid(vector<vector<char>>& board, int row, int col, char c){
        for(int i = 0; i < 9; i++) {
            if(board[i][col] == c)
                return false;

            if(board[row][i] == c)
                return false;

            if(board[3 * (row / 3) + i / 3][3 * (col / 3) + i % 3] == c)
                return false;
        }

        return true;
    }
};
  
```

our previous code was restored from your local storage. [Reset to default](#)

console - [Contribute](#)

All changes saved!